

Νευρωνικά Δίκτυα, 2η Εργασία

Ιωάννης Κέλπης AEM:10300

Δεκέμβριος 2023

Περίληψη

Σκοπός της εργασίας είναι να υλοποιηθεί και να μελετηθεί ένα Support Vector Machine (SVM) σε οποιαδήποτε γλώσσα προγραμματισμού. Η γλώσσα που επιλέχθηκε για το συγκεκριμένο project είναι η Python (version 3.11.5).

Η βάση δεδομένων που επιλέχθηκε είναι η Cifar10, και χρησιμοποιήθηκαν οι 2 από τις 10 κλάσεις της. Πιο συγκεκριμένα η υλοποίηση του SVM, έγινε με την βοήθεια της βιβλιοθήκης scikit-learn, η οποία είναι μια open source βιβλιοθήκη της python, που υποστηρίζει πολλούς αλγορίθμους μηχανικής μάθησης, μέσα σε αυτούς και τα SVM.

Αφού δημιουργήθηκε το SVM, δοκιμάσθηκαν και μελετήθηκαν διάφοροι παράγοντες που επηρεάζουν την ακρίβεια του SVM, όπως ο πυρήνας (kernel) και η παράμετρος C. Στην συνέχεια χρησιμοποιήθηκε και το Grid Search το οποίο είναι μια τεχνική υπερ-παραμετρικής βελτιστοποίησης που χρησιμοποιείται στη μηχανική μάθηση για την αναζήτηση του βέλτιστου συνδυασμού υπερ-παραμέτρων για ένα μοντέλο, προκειμένου να βρούμε τις πιο αποτελεσματικές παραμέτρους. Τέλος παρουσιάζονται και ερμηνεύονται κάποια διαγράμματα που προκύπτουν από τα δεδομένα του προβλήματος, καθώς και η σύγκριση της απόδοσης του SVM με τους κατηγοριοποιητές πλησιέστερου γείτονα και πλησιέστερου κέντρου κλάσης της ενδιάμεσης εργασίας.

Chapter 1

Παρουσίαση και ανάλυση του κώδικα

Σε αυτό το κεφάλαιο θα μελετήσουμε και θα αναλύσουμε τον κώδικα που είναι υπεύθυνος για την δημιουργία και την εκπαίδευση του SVM μας.

Πρώτο και σημαντικότερο βήμα, είναι η εισαγωγή των απαραίτητων βιβλιοθηκών.

- "sklearn": από αυτήν την βιβλιοθήκη εισάγουμε απαραίτητες λειτουργίες για το πρόγραμμα μας, όπως την προεπεξεργασία των δεδομένων, και φυσικά το ίδιο το SVM μοντέλο. Ακόμα με την βοήθεια της υλοποιούμε τις τεχνικές του Grid Search, των διαγραμμάτων εκπαίδευσης και του confusion matrix.
- "matplotlib.pyplot", "seaborn": παρέχουν εργαλεία για την γραφική αναπαράσταση των αποτελεσμάτων του SVM μας.
- "tensorflow": χρησιμοποιείται για την "φόρτωση" των δεδομένων μας.
- "time": βιβλιοθήκη που μας επιτρέπει να "ελέγχουμε" πόσο χρόνο διαρκεί κάποια διεργασία στο πρόγραμμα μας.
- "numpy": βιβλιοθήκη που μας επιτρέπει

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import learning_curve
from sklearn.decomposition import PCA
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
import time
```

Στην συνέχεια, ο κώδικας μας φορτώνει το dataset CIFAR-10 χρησιμοποιώντας το TensorFlow. Αυτό το dataset αποτελείται από 60.000 εικόνες 32x32 χρωματικών επιπέδων σε 10 διαφορετικές κατηγορίες. Στην εργασία μου, αποφάσισα να επιλέξω τις δύο από αυτές, (ενδεικτικά την 3 και 5, δηλαδή τις εικόνες σκύλων και γάτων) και να υλοποιήσω ένα SVM που θα τις διαχωρίζει. Τελικά δηλαδή, έχουμε 10.000 εικόνες για το σετ εκπαίδευσης, και 2.000 για το σετ ελέγχου. Το φιλτράρισμα αυτό των 2 κλάσεων επιτυγχάνεται με τον παρακάτω κώδικα.

```

# Load CIFAR-10 dataset using TensorFlow
(X_train, y_train), (X_test, y_test) =
tf.keras.datasets.cifar10.load_data()

# Choose two classes for binary classification (e.g., cats and dogs)
class1, class2 = 3, 5

# Filter data for the selected classes in both training and test sets
train_class_indices = np.where((y_train == class1) |
(y_train == class2))[0]
test_class_indices = np.where((y_test == class1) |
(y_test == class2))[0]

X_train_binary = X_train[train_class_indices]
y_train_binary = y_train[train_class_indices]
X_test_binary = X_test[test_class_indices]
y_test_binary = y_test[test_class_indices]

```

Στην συνέχεια, κάνουμε "flatten" τα δεδομένα της εικόνας, ώστε να έχουν ένα προκαθορισμένο μέγεθος, να είναι συμβατά με την είσοδο του SVM, και να μειωθεί η πολυπλοκότητα του μοντέλου μας. Έπειτα, κανονικοποιούμε τα χαρακτηριστικά, προκειμένου να έχουν κοινή κλίμακα. Αυτό το βήμα είναι ιδιαίτερα σημαντικό για τα μοντέλα που βασίζονται σε αλγορίθμους που επηρεάζονται από την κλίμακα των χαρακτηριστικών, όπως τα SVM. Η κανονικοποίηση αυτή γίνεται με τη μετατροπή των τιμών των χαρακτηριστικών έτσι ώστε να έχουν μέση τιμή 0 και τυπική απόκλιση 1, με την βοήθεια του Standard Scaler. Την ίδια στιγμή, εφαρμόζουμε PCA (Principal Component Analysis), προκειμένου να μειώσουμε την διαστατικότητα των δεδομένων, χωρίς όμως να χάσουμε σημαντικά τους χαρακτηριστικά. Έτσι επιταχύνουμε και τους αλγορίθμους που θα ακολουθήσουν.

```

# Flatten the image data
X_train_binary = X_train_binary.reshape(X_train_binary.shape[0], -1)
X_test_binary = X_test_binary.reshape(X_test_binary.shape[0], -1)

# Standardize features by removing the mean and scaling to unit
variance
scaler = StandardScaler()
X_train_binary = scaler.fit_transform(X_train_binary)
X_test_binary = scaler.transform(X_test_binary)

#flatten the target variable
y_train_binary = y_train_binary.ravel()
y_test_binary = y_test_binary.ravel()

#Apply PCA for dimensionality reduction
pca = PCA(n_components=0.90)
X_train_binary = pca.fit_transform(X_train_binary)
X_test_binary = pca.transform(X_test_binary)

```

Επόμενο βήμα στον κώδικα μας, είναι η δημιουργία 3 κατηγοριοποιητών SVM. Το κάθε SVM έχει διαφορετικές συναρτήσεις πυρήνα (γραμμική, πολυωνυμική και rbf). Η συνάρτηση πυρήνα χρησιμοποιείται για τη μετατροπή του χώρου χαρακτηριστικών των δεδομένων, επιτρέποντας την ανίχνευση μη γραμμικών σχέσεων μεταξύ των παρατηρήσεων.

Η γενική μορφή της συνάρτησης πυρήνα είναι:

$$K(x_i, x_j) = \varphi(x_i) * \varphi(x_j) \quad K(x_i, x_j) = \varphi(x_i) * \varphi(x_j)$$

όπου K είναι η συνάρτηση πυρήνα, x_i και x_j είναι δύο διανύσματα στον χώρο των χαρακτηριστικών, και φ είναι η συνάρτηση μετασχηματισμού που μετατρέπει τα δεδομένα στον χώρο χαρακτηριστικών. Οι πιο συνηθισμένοι τύποι συναρτήσεων πυρήνα περιλαμβάνουν:

- Γραμμική Συνάρτηση Πυρήνα:

$$K(x_i, x_j) = x_i * x_j$$

Είναι το απλό εσωτερικό γινόμενο δύο διανυσμάτων.

- Πολυωνυμική Συνάρτηση Πυρήνα:

$$K(x_i, x_j) = (x_i * x_j + c)^d$$

Προσθέτει έναν όρο στο εσωτερικό γινόμενο και το υψώνει σε μια δύναμη d .

- Συνάρτηση Πυρήνα Radial Basis Function (RBF):

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

Είναι επίσης γνωστή ως Gaussian kernel. Εισάγει μια γκαουσιανή συνάρτηση στο εσωτερικό γινόμενο.

```
# Create an SVM classifier
svm_classifier = SVC(kernel='linear', C=1.0, random_state=42)
svm_classifier2 = SVC(kernel='poly', C=1.0, random_state=42)
svm_classifier3 = SVC(kernel='rbf', C=1.0, random_state=42)
```

Στο επόμενο κομμάτι του κώδικα, εκπαιδεύονται και ελέγχονται τα 3 μοντέλα με διαφορετικούς πυρήνες που δημιουργήθηκαν προηγουμένως! Πιο συγκεκριμένα, υπολογίζετε ο χρόνος που παίρνει σε κάθε SVM να εκπαιδευτεί, πραγματοποιούνται προβλέψεις στο σύνολο ελέγχου (test set) και τελικά υπολογίζεται η ακρίβεια του κάθε μοντέλου και εκτυπώνεται.

```

# Train the classifiers and measure training time
start_time = time.time()
svm_classifier.fit(X_train_binary, y_train_binary)
end_time = time.time()
print(f"Training-time-(linear-kernel):
{end_time--start_time:.2f}-seconds")

start_time = time.time()
svm_classifier2.fit(X_train_binary, y_train_binary)
end_time = time.time()
print(f"Training-time-(polynomial-kernel):
{end_time--start_time:.2f}-seconds")

start_time = time.time()
svm_classifier3.fit(X_train_binary, y_train_binary)
end_time = time.time()
print(f"Training-time-(rbf-kernel):
{end_time--start_time:.2f}-seconds")

# Make predictions on the test set
y_pred1 = svm_classifier.predict(X_test_binary)
y_pred2 = svm_classifier2.predict(X_test_binary)
y_pred3 = svm_classifier3.predict(X_test_binary)

# Evaluate the accuracy
accuracy1 = accuracy_score(y_test_binary, y_pred1)
accuracy2 = accuracy_score(y_test_binary, y_pred2)
accuracy3 = accuracy_score(y_test_binary, y_pred3)
print(f"Accuracy-with-linear-kernel:-{accuracy1*100:.2f}%")
print(f"Accuracy-with-polynomial-kernel:-{accuracy2*100:.2f}%")
print(f"Accuracy-with-radial-basis-function-(rbf):
{accuracy3*100:.2f}%")

```

Στην συνέχεια του κώδικα, εκτελούμε ένα Grid Search για την εύρεση των βέλτιστων υπερπαραμέτρων του SVM, δηλαδή το πρόγραμμα μας, εξετάζει συγκεκριμένους συνδυασμούς τιμών για διάφορες υπερπαραμέτρους που έχουν καθοριστεί εκ των προτέρων. Οι συνδυασμοί αυτοί σχηματίζουν ένα "πλέγμα" (grid), και για κάθε συνδυασμό εκπαιδεύεται και αξιολογείται το μοντέλο. Στην περίπτωση μας, εξετάζονται η παράμετρος κόστους C ο πυρήνας του SVM, και κάποιες επιμέρους παράμετροι του κάθε πυρήνα. Αρχικά ορίζεται ο χώρος των υπερπαραμέτρων που θα εξεταστούν στο Grid Search. Συγκεκριμένα, εξετάζονται τρεις πυρήνες (γραμμικός, RBF, πολυωνυμικός), τρεις τιμές για την παράμετρο C (0.05, 0.1, 1, 10) και στην συνέχεια της εργασίας κάποιες τιμές για τις παραμέτρους degree (polynomial kernel) και gamma (rbf kernel). Έπειτα δημιουργείται ένα αντικείμενο GridSearchCV χρησιμοποιώντας ένα μοντέλο SVM με τυχαίο seed (random_state=42) και τον ορισμένο χώρο υπερπαραμέτρων. Η διαδικασία αυτή θα εκτελεστεί με cross-validation (cv=3) για να εκτιμηθεί η απόδοση των μοντέλων με διάφορες υπερπαραμέτρους. Το Grid Search εκτελείται πάνω στα δεδομένα εκπαίδευσης, εκπαιδεύοντας και αξιολογώντας τα μοντέλα για όλους τους συνδυασμούς υπερπαραμέτρων. Τελικά, οι υπερπαραμέτροι του μοντέλου που επιτυγχάνει την υψηλότερη απόδοση αποθηκεύονται και χρησιμοποιούνται για την δημιουργία του βέλτιστου μοντέλου, του οποίου η απόδοση εκτιμάται πάνω στο σύνολο ελέγχου. Να σημειωθεί ότι την ίδια στιγμή, υπολογίζουμε και τον χρόνο που πήρε στο πρόγραμμα μας να πραγματοποιήσει το Grid Search. *Το κομμάτι αυτό του κώδικα το προσαρμόζουμε ανάλογα με τις ανάγκες μας στην έκταση της εργασίας, προκειμένου να βγάλουμε τα αντίστοιχα συμπεράσματα και να μελετήσουμε διαφορετικές παραμέτρους.*

```

# Grid Search
param_grid = {'kernel': ['linear', 'rbf', 'poly'],
              'C': [0.05, 0.1, 1, 10]}
grid_search = GridSearchCV(SVC(random_state=42), param_grid, cv=3)

# Measure grid search time
start_time = time.time()
grid_search.fit(X_train_binary, y_train_binary)
end_time = time.time()

# Get the best hyperparameters
best_params = grid_search.best_params_
print("Best Hyperparameters:", best_params)

# Get the best model
best_svm_classifier = grid_search.best_estimator_

# Evaluate the best model on the test set
y_pred_best = best_svm_classifier.predict(X_test_binary)
accuracy_best = accuracy_score(y_test_binary, y_pred_best)
print(f"Best Model Accuracy on Test Set: {accuracy_best * 100:.2f}%")

print(f"Grid Search time: {end_time - start_time:.2f} seconds")

```

Στο επόμενο κομμάτι του κώδικα, υλοποιούμε τις καμπύλες εκπαίδευσης, προκειμένου να αξιολογήσουμε γραφικά την συμπεριφορά του μοντέλου μας όσο μεγαλώνει ο όγκος των δεδομένων στα οποία έχει εκπαιδευτεί. Αυτές θα δούμε στην συνέχεια ότι δίνουν κάποιες ενδιαφέροντες πληροφορίες για την συμπεριφορά του SVM μας.

```

# Plotting the learning curves
train_sizes , train_scores , test_scores =
learning_curve(svm_classifier3 , X_train_binary , y_train_binary , cv=5)
train_scores_mean = np.mean(train_scores , axis=1)
test_scores_mean = np.mean(test_scores , axis=1)

plt.figure()
plt.plot(train_sizes , train_scores_mean , label='Training-score')
plt.plot(train_sizes , test_scores_mean , label='Cross-validation-score')
plt.xlabel('Training-examples')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# Plotting the learning curves
train_sizes , train_scores , test_scores =
learning_curve(svm_classifier3 , X_train_binary , y_train_binary , cv=5)
train_scores_mean = np.mean(train_scores , axis=1)
test_scores_mean = np.mean(test_scores , axis=1)

plt.figure()
plt.plot(train_sizes , train_scores_mean , label='Training-score')
plt.plot(train_sizes , test_scores_mean , label='Cross-validation-score')
plt.xlabel('Training-examples')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

```

Προκειμένου να κατανοήσουμε καλύτερα τα αποτελέσματά μας, υλοποιούμε και έναν Confusion Matrix ο οποίος αξιολογεί το μοντέλο μας (χρησιμοποιήσαμε το βέλτιστο μοντέλο που βρήκαμε προηγουμένως). Το πετυχαίνουμε αυτό για ακόμη μια φορά με την βοήθεια της βιβλιοθήκης sklearn και την συνάρτηση confusion_matrix. Πιο αναλυτική αναφορά στον Confusion Matrix θα γίνει στο επόμενο κομμάτι της εργασίας.

```

# Confusion Matrix
conf_matrix = confusion_matrix(y_test_binary , y_pred_best)
print("Confusion-Matrix:")
print(conf_matrix)

# Plotting the confusion matrix
class_names = [f"Class-{class1}" , f"Class-{class2}"]
plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix , annot=True , fmt='d' ,
cmap='Blues' , xticklabels=class_names , yticklabels=class_names)
plt.title('Confusion-Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()

```

Στο τελευταίο κομμάτι του κώδικα μας, εκτυπώνουμε μερικά παραδείγματα ορθής και εσφαλμένης κατηγοριοποίησης.

```

# Visualizing some right and wrong examples
correct_indices = np.where(y_test_binary == y_pred_best)[0]
incorrect_indices = np.where(y_test_binary != y_pred_best)[0]

# Map class labels to indices
label_to_index = {class1: 0, class2: 1}

# Display 10 correct examples
plt.figure(figsize=(20, 8))
for i, idx in enumerate(correct_indices[:10]):
    plt.subplot(2, 10, i + 1)
    plt.imshow(X_test[test_class_indices][idx])
    plt.title(f'True:
----{class_names[label_to_index[y_test_binary[idx]]]}
----\nPred:
----{class_names[label_to_index[y_pred_best[idx]]]} ')
    plt.axis('off')

# Display 10 incorrect examples
for i, idx in enumerate(incorrect_indices[:10]):
    plt.subplot(2, 10, i + 11)
    plt.imshow(X_test[test_class_indices][idx])
    plt.title(f'True:
----{class_names[label_to_index[y_test_binary[idx]]]} \nPred:
----{class_names[label_to_index[y_pred_best[idx]]]} ')
    plt.axis('off')

plt.show()

```

Chapter 2

Έκθεση των αποτελεσμάτων

Σε αυτό το κεφάλαιο, θα παρουσιαστούν κάποια αποτελέσματα που προέκυψαν από την δημιουργία και την μελέτη του SVM μας. Αναλυτικότερα θα γίνει αναφορά:

- σε μερικά παραδείγματα ορθής και εσφαλμένης κατηγοριοποίησης
- στα ποσοστά επιτυχίας του SVM μας, για διαφορετικές τιμές του C και άλλων παραμέτρων, καθώς και για διαφορετικές συναρτήσεις πυρήνα.
- στον χρόνο που χρειάζεται το SVM μας για να εκπαιδευτεί, καθώς και για άλλες διεργασίες που λαμβάνουν χώρα στον κώδικα μας.
- στην ανάλυση και την παρουσίαση των αποτελεσμάτων του Confusion Matrix
- στην παρουσίαση και την ερμηνεία των learning curves
- στην ταξινόμηση διαφορετικών κλάσεων

Τέλος, θα γίνει εν συντομία και μια σύγκριση μεταξύ της απόδοσης του SVM, σε σχέση με τους κατηγοριοποιητές πλησιέστερου γείτονα (KNN) και πλησιέστερου κέντρου (Nearest Centroid) της ενδιάμεσης εργασίας.

Na σημειωθεί, πως στην υλοποίηση μας το dataset που χρησιμοποιήθηκε ήταν η *Cifar10*, από την οποία διαχωρίζουμε μόνο τις κλάσεις 3 και 5, δηλαδή εικόνες σκύλων και γατών.

2.1 Ορθή - Εσφαλμένη κατηγοριοποίηση

Παρακάτω, παρουσιάζονται 10 παραδείγματα ορθής, και 10 παραδείγματα εσφαλμένης κατηγοριοποίησης. Το SVM που έκανε αυτές τις προβλέψεις, είχε όπως φαίνεται στην 2η φωτογραφία, ποσοστό ακριβείας 63.4% και οι παράμετροι του ήταν $C = 1$, $\text{kernel} = \text{poly}$, $\text{degree} = 3$.

```
Fitting 3 folds for each of 2 candidates, totalling 6 fits
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.5s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.5s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.5s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.4s
Best Hyperparameters: {'C': 1, 'degree': 3, 'kernel': 'poly'}
Best Model Accuracy on Test Set: 63.40%
```

Figure 2.1: Output

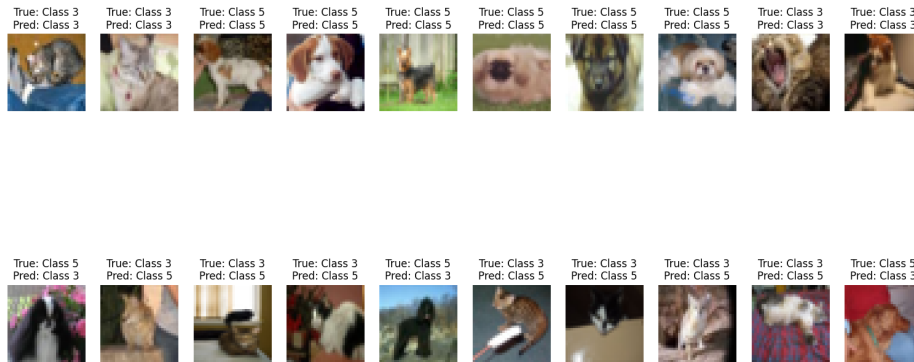


Figure 2.2: Classification Images

2.2 Ακρίβεια και χρόνος του SVM για διαφορετικές συναρτήσεις πυρήνα

Στο παρακάτω πίνακά, παραθέτονται ο χρόνος εκπαίδευσης του SVM, καθώς και η ακρίβεια που πετυχαίνουμε για διαφορετικές συναρτήσεις πυρήνα και $C = 1$.

Ας εξηγήσουμε αρχικά εν συντομία την παράμετρο C . Αυτή αναφέρεται στην παράμετρο κόστους και ελέγχει τα όρια μεταξύ μια ομαλής επιφάνειας αποφάσεων και της ακρίβειας της ταξινόμησης στα δεδομένα εκπαίδευσης. Όταν η τιμή του C είναι υψηλή, το SVM θα προσπαθήσει να ταξινομήσει όσα περισσότερα δεδομένα εκπαίδευσης μπορεί σωστά, ακόμη και αν αυτό σημαίνει ότι η απόφαση του μοντέλου είναι πολύ πολύπλοκη και μπορεί να προσαρμοστεί ακόμη και σε θόρυβο ή ατυχή δεδομένα. Αυτό μπορεί να οδηγήσει σε overfitting στα δεδομένα εκπαίδευσης. Αντίθετα, όταν η τιμή του C είναι χαμηλή, το SVM θα επιλέξει μια απλούστερη απόφαση που ενδέχεται να μην ταξινομεί σωστά όλα τα δεδομένα εκπαίδευσης, αλλά προσπαθεί να δημιουργήσει έναν γενικό κανόνα που ισχύει και για νέα, μη ορατά δεδομένα (αποφεύγοντας το overfitting).

Παρατηρούμε πως ο γραμμικός πυρήνας παίρνει πολύ παραπάνω χρόνο από τους άλλους δύο για να εκπαιδευτεί. Αυτό μπορεί να συμβεί για διάφορους λόγους. Τα δεδομένα που προσπαθούμε να ταξινομήσουμε, ίσως είναι περίπλοκα και όχι γραμμικά διαχωρίσιμα, με αποτέλεσμα να απαιτείται περισσότερος χρόνος εκπαίδευσης από τον γραμμικό πυρήνα. Ακόμα, ο όγκος των δεδομένων μπορεί να επηρεάσει σημαντικά τον χρόνο όπως και η επιλογή των παραμέτρων που ίσως να μην είναι ιδανική και δημιουργεί και αυτή μια καθυστέρηση.

Είναι φανερό πως ο πυρήνας rbf έχει μεγαλύτερο ποσοστό ακριβείας από τους άλλους δύο. Η επιτυχία αυτή μπορεί να οφείλεται στην ικανότητα του να προσαρμόζεται σε πιο περίπλοκες δομές δεδομένων. Η RBF συνάρτηση πυρήνα, είναι γνωστή για την ικανότητα της να προσαρμόζεται σε μη γραμμικές κατανομές δεδομένων, επιτρέποντας το SVM να δημιουργήσει πιο περίπλοκες αποφάσεις αποκλεισμού.

kernel	Training Time	Accuracy
linear	218.31s	61.15%
polynomial	4.48s	63.4%
rbf	4.15s	66.55%

Table 2.1

Θα μελετήσουμε λίγο ακόμα την συνάρτηση πυρήνα RBF, μιας και αυτή μας έδωσε τα καλύτερα αποτελέσματα όπως είδαμε παραπάνω. Το kernel RBF, έχει κάποια ακόμα παράμετρο την οποία μπορούμε

να διερευνήσουμε, την παράμετρο γ (gamma). Η παράμετρος αυτή, καθορίζει το πόσο "ελαστικό" είναι το πεδίο επιδόσεων του πυρήνα. Επηρεάζει τον τρόπο με τον οποίο το SVM προσαρμόζει τις αποφάσεις αποκλεισμού στα δεδομένα εκπαίδευσης. Ειδικότερα:

- Χαμηλή τιμή του γ : Εκτείνει το πεδίο επιδόσεων και καθιστά τον πυρήνα RBF πιο "ελαστικό", με αποτέλεσμα το μοντέλο να είναι λιγότερο ευαίσθητο στις λεπτομέρειες των δεδομένων εκπαίδευσης.
- Υψηλή τιμή του γ : Σφίγγει το πεδίο επιδόσεων και καθιστά τον πυρήνα RBF πιο "σφιχτό", με αποτέλεσμα το μοντέλο να είναι πιο ευαίσθητο στις λεπτομέρειες και τον θόρυβο των δεδομένων εκπαίδευσης.

Θα προσπαθήσουμε λοιπόν να εξετάσουμε την επίδραση της παραμέτρου αυτής στο SVM μας, και να βρούμε την βέλτιστη τιμή που μπορεί να πάρει. Για να το καταφέρουμε αυτό, θα υλοποιήσουμε μια μικρής κλίμακας grid search.

```
# Grid Search
parameters_grid = {'kernel': [ 'rbf'],
                   'C': [1], 'gamma':[0.05, 0.1, 0.5, 1.0, 2.0]}
grid_search = GridSearchCV(SVC(random_state=42), parameters_grid, cv=3, scoring='accuracy', verbose = 2)

# Measure grid search time
start_time = time.time()
grid_search.fit(X_train_binary, y_train_binary)
end_time = time.time()

# Get the best hyperparameters
best_params = grid_search.best_params_
print("Best Hyperparameters:", best_params)

# Get the best model
best_svm_classifier = grid_search.best_estimator_
```

Figure 2.3: Grid Search

Κρατάμε σταθερή την παράμετρο C και ίση με 1, και αλλάζουμε τις τιμές του gamma. Δοκιμάζουμε τις τιμές 0.05, 0.1, 0.5, 1.0, 2.0 αντίστοιχα.

```
Fitting 3 folds for each of 5 candidates, totalling 15 fits
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.7s
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.6s
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 4.5s
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 4.5s
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 4.5s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 4.9s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 4.9s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 5.0s
[CV] END .....C=1, gamma=1, kernel=rbf; total time= 4.9s
[CV] END .....C=1, gamma=1, kernel=rbf; total time= 4.9s
[CV] END .....C=1, gamma=1, kernel=rbf; total time= 4.9s
[CV] END .....C=1, gamma=2, kernel=rbf; total time= 4.9s
[CV] END .....C=1, gamma=2, kernel=rbf; total time= 4.9s
[CV] END .....C=1, gamma=2, kernel=rbf; total time= 4.8s
Best Hyperparameters: {'C': 1, 'gamma': 2, 'kernel': 'rbf'}
Best Model Accuracy on Test Set: 50.00%
```

Figure 2.4: Results

Όπως φαίνεται από τα αποτελέσματα, το SVM μας αποκτά την μέγιστη ακρίβεια 50.00% για $\gamma = 2$. Εφόσον είναι η μέγιστη τιμή που είχαμε στο grid μας, αποφάσισα να φτιάξω άλλο ένα πλέγμα στο οποίο θα ξεκινάμε από αυτήν την τιμή γ και θα φτάνουμε μέχρι την τιμή 100 προκειμένου να βρεθεί ενδεχομένως μια καλύτερη τιμή. Πιο συγκεκριμένα δοκιμάστηκαν οι τιμές $\gamma = [2.0, 5.0, 10.0, 20.0, 50.0, 100.0]$. Το αποτέλεσμα του grid search φαίνεται να δείχνει πως το βέλτιστο γ είναι και πάλι η τιμή 2.0! Μάλιστα, διαπιστώνουμε πως το μοντέλο μας κατατάσσει όλες τις εικόνες μας στην class 5. Για αυτόν τον λόγο έχουμε ακρίβεια 50%!

Προκειμένου να διερευνήσουμε παραπάνω γιατί συμβαίνει αυτό, αλλάζουμε το grid search, και βάζουμε πλέον τις τιμές 'C': [0.1, 1, 10], 'gamma': [0.1, 1, 10]. Το πρόβλημα επιμένει! Το μοντέλο μας συνεχίζει να έχει βέλτιστη ακρίβεια το 50%, αυτή την φορά όμως για gamma = 1, και C = 0.1!

Προκειμένου να επιβεβαιώσω αν το γεγονός αυτό έχει να κάνει με την rbf, θα πραγματοποιήσουμε grid search αυτή την φορά με συνάρτηση πυρήνα την polynomial. Το πλέγμα λοιπόν που δημιουργούμε είναι το εξής: parameters_grid = 'kernel': ['poly'], 'C': [0.1, 1, 10], 'degree': [1, 2, 3, 4]. Όπως φαίνεται παρακάτω, για kernel = polynomial, το grid search δουλεύει κανονικά, και το μοντέλο μας ταξινομεί σωστά τα δεδομένα!

```
Fitting 3 folds for each of 12 candidates, totalling 36 fits
[CV] END .....C=0.1, degree=1, kernel=poly; total time= 2.0s
[CV] END .....C=0.1, degree=1, kernel=poly; total time= 2.0s
[CV] END .....C=0.1, degree=1, kernel=poly; total time= 2.0s
[CV] END .....C=0.1, degree=2, kernel=poly; total time= 2.1s
[CV] END .....C=0.1, degree=2, kernel=poly; total time= 2.2s
[CV] END .....C=0.1, degree=2, kernel=poly; total time= 2.2s
[CV] END .....C=0.1, degree=3, kernel=poly; total time= 2.2s
[CV] END .....C=0.1, degree=3, kernel=poly; total time= 2.2s
[CV] END .....C=0.1, degree=3, kernel=poly; total time= 2.2s
[CV] END .....C=0.1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=0.1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=0.1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=1, degree=1, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=1, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=1, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.2s
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.2s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.5s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.5s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.4s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.4s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=10, degree=1, kernel=poly; total time= 2.8s
[CV] END .....C=10, degree=1, kernel=poly; total time= 2.8s
[CV] END .....C=10, degree=1, kernel=poly; total time= 2.7s
[CV] END .....C=10, degree=2, kernel=poly; total time= 2.9s
[CV] END .....C=10, degree=2, kernel=poly; total time= 2.9s
[CV] END .....C=10, degree=2, kernel=poly; total time= 3.0s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.6s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.8s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.9s
[CV] END .....C=10, degree=4, kernel=poly; total time= 2.8s
[CV] END .....C=10, degree=4, kernel=poly; total time= 2.9s
[CV] END .....C=10, degree=4, kernel=poly; total time= 3.0s
Best Hyperparameters: {'C': 10, 'degree': 2, 'kernel': 'poly'}
Best Model Accuracy on Test Set: 63.45%
```

Figure 2.5: Polynomial Grid

Έπειτα από περαιτέρω μελέτη του προβλήματος, κατέληξα στο συμπέρασμα ότι το gamma που χρησιμοποιούνταν στις παραπάνω δοκιμές ήταν πολύ μεγάλο, και για αυτό το μοντέλο μας δεν λειτουργούσε σωστά! Αυτό φαίνεται από τα αποτελέσματα της grid search για 'kernel': ['rbf'], 'C': [1], 'gamma': [0.001, 0.02, 0.5, 1, 10, 100, 1000].

```

Fitting 3 folds for each of 7 candidates, totalling 21 fits
[CV] END .....C=1, gamma=0.001, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=0.001, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.001, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.02, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.02, kernel=rbf; total time= 3.7s
[CV] END .....C=1, gamma=0.02, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 4.8s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 4.8s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 4.8s
[CV] END .....C=1, gamma=1, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=1, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=1, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=10, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=10, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=10, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=100, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=100, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=100, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=1000, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=1000, kernel=rbf; total time= 4.6s
[CV] END .....C=1, gamma=1000, kernel=rbf; total time= 4.7s
Best Hyperparameters: {'C': 1, 'gamma': 0.001, 'kernel': 'rbf'}
Best Model Accuracy on Test Set: 67.00%

```

Figure 2.6: Grid Results

Το μοντέλο μας αποκτάει την μέγιστη ακρίβεια για $\gamma = 0,001$! Θα κάνουμε μια τελευταία grid search προκειμένου να βρούμε την τελική βέλτιστη τιμή της γ . Για 'gamma': [0.0001, 0.0002, 0.0005, 0.001], βλέπουμε πως το SVM αποκτάει και πάλι την μέγιστη ακρίβεια 67.00% όταν $\gamma = 0.001$. Επομένως, κρατάμε αυτή την τιμή, και έχουμε έτσι το SVM με την μέγιστη ακρίβεια!

```

Fitting 3 folds for each of 4 candidates, totalling 12 fits
[CV] END .....C=1, gamma=0.0001, kernel=rbf; total time= 3.0s
[CV] END .....C=1, gamma=0.0001, kernel=rbf; total time= 3.1s
[CV] END .....C=1, gamma=0.0001, kernel=rbf; total time= 3.0s
[CV] END .....C=1, gamma=0.0002, kernel=rbf; total time= 3.0s
[CV] END .....C=1, gamma=0.0002, kernel=rbf; total time= 3.1s
[CV] END .....C=1, gamma=0.0002, kernel=rbf; total time= 3.0s
[CV] END .....C=1, gamma=0.0005, kernel=rbf; total time= 3.2s
[CV] END .....C=1, gamma=0.0005, kernel=rbf; total time= 3.2s
[CV] END .....C=1, gamma=0.0005, kernel=rbf; total time= 3.2s
[CV] END .....C=1, gamma=0.001, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=0.001, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=0.001, kernel=rbf; total time= 3.5s
Best Hyperparameters: {'C': 1, 'gamma': 0.001, 'kernel': 'rbf'}
Best Model Accuracy on Test Set: 67.00%

```

Figure 2.7: Grid Results

Παρακάτω παρουσιάζονται ο confusion matrix και τα learning curves (βλ. παρακάτω για περισσότερες πληροφορίες) του βέλτιστου μοντέλου!

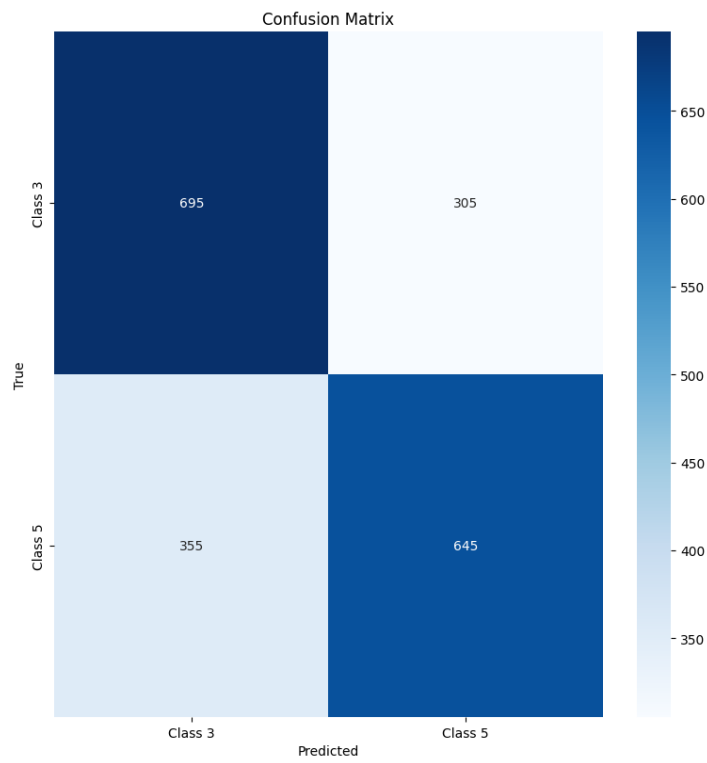


Figure 2.8: Confusion Matrix

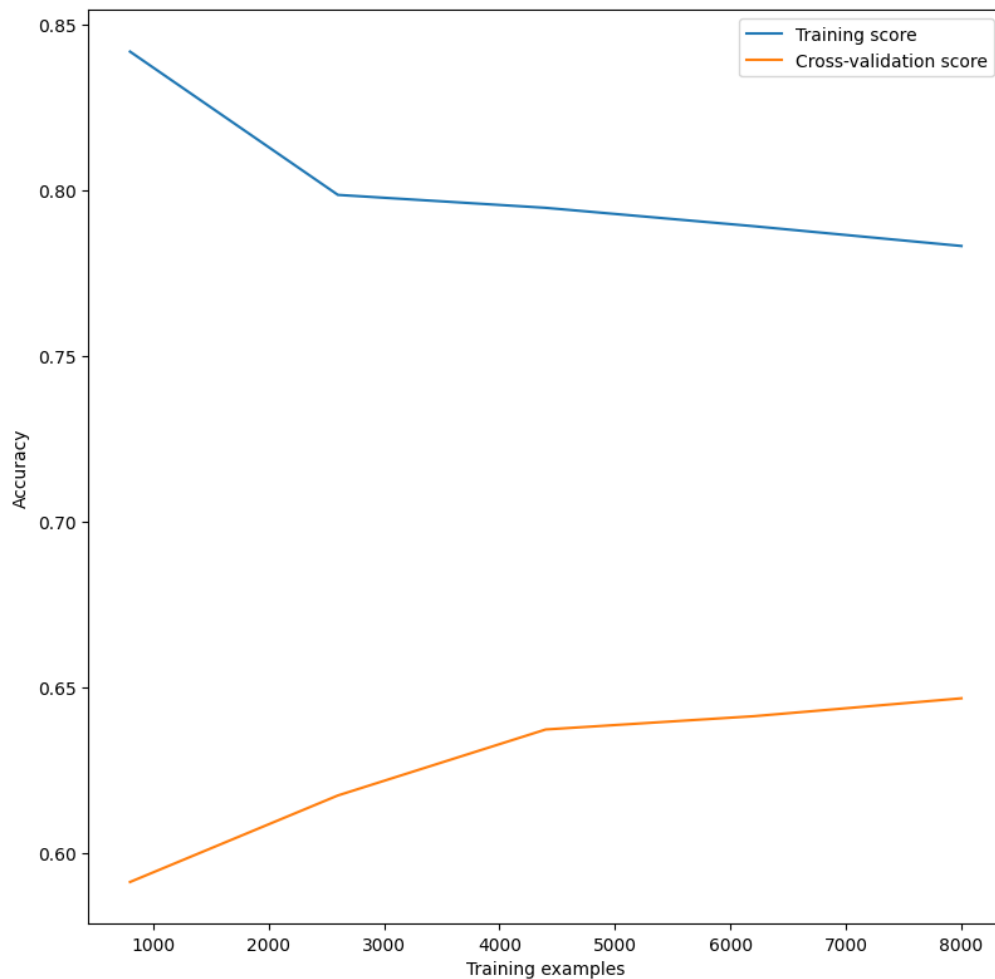


Figure 2.9: Learning Curves

Θα πραγματοποιήσουμε την ίδια διαδικασία για το polynomial kernel και την παράμετρο degree αντίστοιχα. Αυτή, καθορίζει τον βαθμό του πολυωνυμικού πυρήνα. Ο πυρήνας πολυωνύμου έχει τη μορφή: $K(x, y) = (x * y + c)^d$ όπου x, y είναι τα δύο διανύσματα εισόδου, και c είναι μια σταθερά. Το d είναι ο βαθμός του πολυωνύμου. Όσο μεγαλύτερη η τιμή του degree, τόσο πιο υψηλή είναι η τάξη του πολυωνύμου και τόσο πιο πολύπλοκος είναι ο χώρος των χαρακτηριστικών που το μοντέλο μπορεί να αντιπροσωπεύσει. Ωστόσο, υψηλότερες τιμές του degree μπορεί να οδηγήσουν σε μοντέλα που χρειάζονται περισσότερα δεδομένα εκπαίδευσης και είναι πιο επιρρεπή στην υπερεκπαίδευση. Το πλέγμα που θα υλοποιήσουμε φαίνεται παρακάτω. Δοκιμάζουμε δύο τιμές της C και τρεις διαφορετικές τιμές του degree.

```
# Grid Search
parameters_grid = {'kernel': ['poly'],
                   'C': [1, 10],
                   'degree': [2, 3, 4]}
grid_search = GridSearchCV(SVC(random_state=42), parameters_grid, cv=3, scoring='accuracy', verbose = 2)
```

Figure 2.10: Poly Grid

```

Fitting 3 folds for each of 6 candidates, totalling 18 fits
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.4s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.5s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.5s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.4s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.3s
[CV] END .....C=10, degree=2, kernel=poly; total time= 2.9s
[CV] END .....C=10, degree=2, kernel=poly; total time= 2.9s
[CV] END .....C=10, degree=2, kernel=poly; total time= 3.0s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.6s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.8s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.7s
[CV] END .....C=10, degree=4, kernel=poly; total time= 2.7s
[CV] END .....C=10, degree=4, kernel=poly; total time= 3.0s
[CV] END .....C=10, degree=4, kernel=poly; total time= 3.0s
Best Hyperparameters: {'C': 10, 'degree': 2, 'kernel': 'poly'}
Best Model Accuracy on Test Set: 63.45%

```

Figure 2.11: Poly Results

Από τα αποτελέσματα, φαίνεται πως το kernel polynomial δεν έχει την ίδια απόδοση με το rbf. Καταλήγουμε έτσι λοιπόν πως η ιδανικότερη συνάρτητη πυρήνα για τον τύπο των δεδομένων μας και για το SVM μας είναι η rbf.

2.3 Confusion Matrix

Ο Confusion Matrix ή αλλιώς Πίνακας Σύγχυσης, είναι ένας πίνακας που χρησιμοποιείται στον τομέα της μηχανικής μάθησης, για να αξιολογήσει την απόδοση ενός μοντέλου. Παρέχει μια συνοπτική έκθεση των αποτελεσμάτων όσον αφορά τις προβλέψεις του SVM μας και τις πραγματικές κλάσεις στις οποίες ανήκουν τα δεδομένα. Ο Confusion Matrix έχει τέσσερις πιθανές καταστάσεις:

1. True Positive (TP): Ο αριθμός των δειγμάτων που ανήκουν στην κλάση 1 και η πρόβλεψη του μοντέλου είναι σωστή.
2. True Negative (TN): Ο αριθμός των δειγμάτων που ανήκουν στην κλάση 2 και η πρόβλεψη του μοντέλου είναι σωστή.
3. False Positive (FP): Ο αριθμός των δειγμάτων που ανήκουν στην κλάση 2, αλλά η πρόβλεψη του μοντέλου είναι λανθασμένα στην κλάση 1.
4. False Negative (FN): Ο αριθμός των δειγμάτων που ανήκουν στη κλάση 1, αλλά η πρόβλεψη του μοντέλου είναι λανθασμένα στην κλάση 2.

Παρακάτω παραθέτουμε την μορφή που έχει ένας πίνακας σύγχυσης (όπου Positive = κλάση 1, Negative = κλάση 2).

	Predicted Positive	Predicted Negative
Actual positive	TP	FN
Actual negative	FP	TN

Table 2.2: Confusion Matrix

Ενδεικτικά, ο Confusion Matrix του SVM με παραμέτρους 'C': 10 'kernel': 'polynomial' degree: 2 είναι ο παρακάτω:

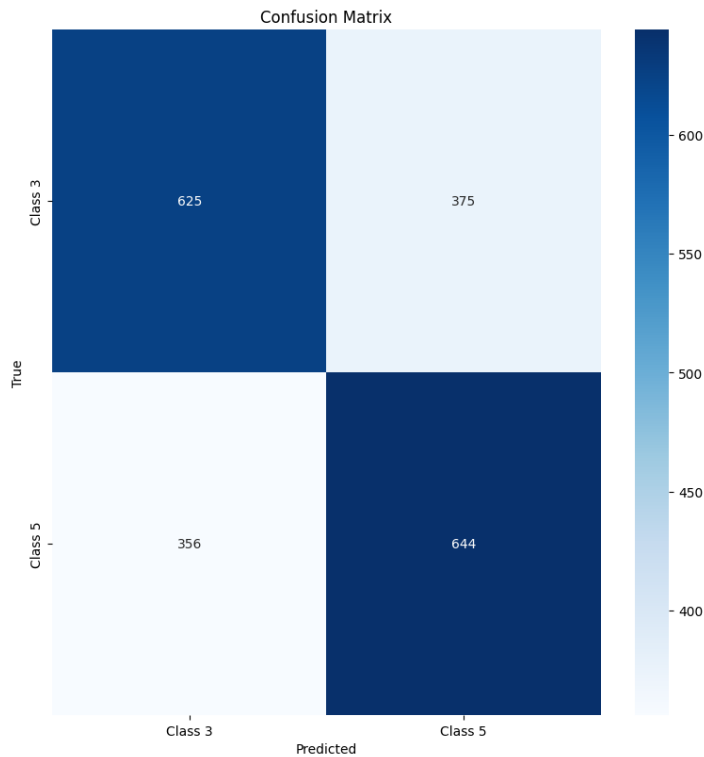


Figure 2.12: Confusion Matrix polynomial

2.4 Learning Curves - Καμπύλες μάθησης

Τα Learning Curves αντιπροσωπεύουν γραφικά την απόδοση του μοντέλου μας σε σχέση με τον όγκο των δεδομένων εκπαίδευσης. Πιο συγκεκριμένα, οι δύο καμπύλες που απεικονίζονται, "μεταφράζονται" ως εξής:

- Καμπύλη Εκπαίδευσης (Training Curve): Αντιπροσωπεύει την επίδοση του μοντέλου στα δεδομένα εκπαίδευσης καθώς αυξάνεται ο όγκος των δεδομένων.
- Καμπύλη Επικύρωσης (Validation Curve): Αντιπροσωπεύει την επίδοση του μοντέλου σε ένα ανεξάρτητο σύνολο δεδομένων επικύρωσης (ή συχνά σε μια στρατηγική διασποράς) καθώς αυξάνεται ο όγκος των δεδομένων, δηλαδή πόσο καλό είναι το generalization του μοντέλου.

Αν οι δύο καμπύλες συγκλίνουν και διατηρούν χαμηλό σφάλμα (ή υψηλή ακρίβεια), αυτό υποδηλώνει ότι το μοντέλο συνεχίζει να βελτιώνεται και δεν υπάρχει υπερεκπαίδευση.

Αν οι δύο καμπύλες αποκλίνουν αυτό υποδηλώνει πιθανή υπερεκπαίδευση. Το μοντέλο μάθησης μπορεί να προσαρμόζεται υπερβολικά στα δεδομένα εκπαίδευσης, αλλά δυσκολεύεται να γενικεύει σε νέα δεδομένα. Παρακάτω, παρατίθενται τα Learning Curves του SVM με παραμέτρους 'C': 1, 'kernel': 'rbf', gamma = 0.001

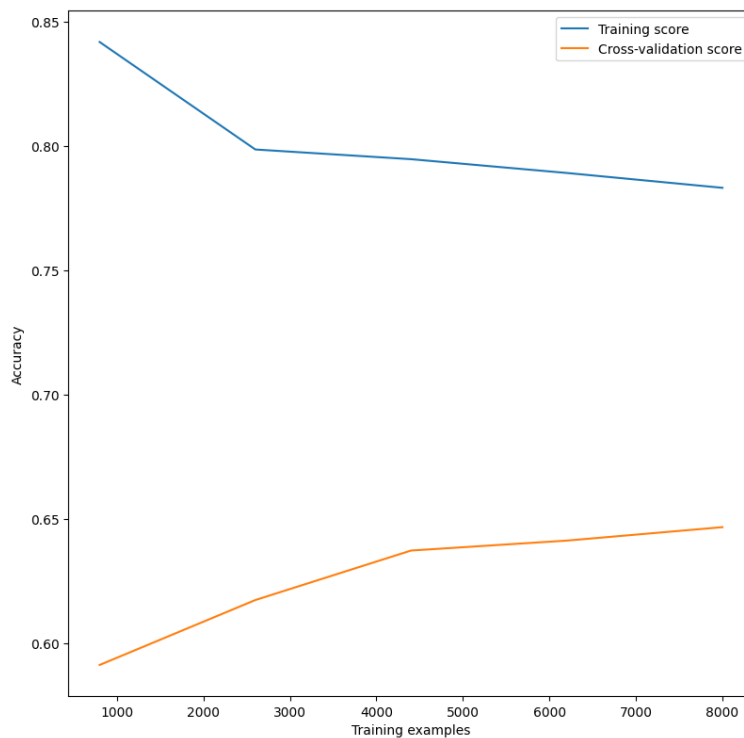


Figure 2.13: Learning Curves

Αυτό που παρατηρούμε είναι πως η ακρίβεια στο training set μειώνεται όσο μεγαλώνει ο όγκος των δεδομένων. Αυτό μπορεί να συμβαίνει διότι όσο μεγαλώνουν τα δεδομένα, το SVM δυσκολεύεται να τα ταξινομήσει ικανοποιητικά με αποτέλεσμα να μειώνεται η ακρίβεια του.

Αντίθετα, το validation curve φαίνεται να αυξάνεται, δηλαδή μεγαλώνει η ακρίβεια του μοντέλου μας στο test set. Αυτό σημαίνει ότι το μοντέλο μας κάνει καλή δουλειά στην ταξινόμηση των νέων δεδομένων.

Γενικότερα, το γεγονός ότι οι καμπύλες μας τείνουν να συγκλίνουν και να σταθεροποιηθούν σε μια τιμή, υποδηλώνει ότι έχει γίνει μια ικανοποιητική επιλογή των υπερπαραμέτρων και ότι το SVM μας δουλεύει σωστά!

2.5 Ταξινόμηση διαφορετικών κλάσεων

Στην συγκεκριμένη ενότητα, γίνεται μια γρήγορη αναφορά στην ταξινόμηση διαφορετικών κλάσεων πέραν από τις 3 και 5 (γάτες και σκύλοι αντίστοιχα). Πιο συγκεκριμένα, η μοναδική αλλαγή που χρειάστηκε να γίνει στον κώδικα ήταν να αλλάξουμε το νούμερο 3 σε 1 στο κομμάτι του κώδικα που φιλτράρουμε τα δεδομένα. Με αυτό τον τρόπο, ταξινομούμε πλέον εικόνες αυτοκινήτων και σκύλων. Όπως μπορούμε να δούμε παρακάτω, τα SVM μας, εκπαιδεύονται πιο γρήγορα αυτή την φορά, και έχουν πολύ υψηλά ποσοστά ακριβείας!

```

Training time (polynomial kernel): 3.13 seconds
Training time (rbf kernel): 2.09 seconds
The gamma value used in the svm_classifier3 is scale.
Accuracy with polynomial kernel: 85.90%
Accuracy with radial basis function (rbf): 92.90%
The gamma value used in the svm_classifier3 is scale.
Fitting 3 folds for each of 6 candidates, totalling 18 fits
[CV] END .....C=1, degree=2, kernel=poly; total time= 1.6s
[CV] END .....C=1, degree=2, kernel=poly; total time= 1.6s
[CV] END .....C=1, degree=2, kernel=poly; total time= 1.5s
[CV] END .....C=1, degree=3, kernel=poly; total time= 1.8s
[CV] END .....C=1, degree=3, kernel=poly; total time= 1.8s
[CV] END .....C=1, degree=3, kernel=poly; total time= 1.8s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.1s
[CV] END .....C=10, degree=2, kernel=poly; total time= 1.5s
[CV] END .....C=10, degree=2, kernel=poly; total time= 1.5s
[CV] END .....C=10, degree=2, kernel=poly; total time= 1.5s
[CV] END .....C=10, degree=3, kernel=poly; total time= 1.8s
[CV] END .....C=10, degree=3, kernel=poly; total time= 1.8s
[CV] END .....C=10, degree=3, kernel=poly; total time= 1.8s
[CV] END .....C=10, degree=4, kernel=poly; total time= 1.8s
[CV] END .....C=10, degree=4, kernel=poly; total time= 1.8s
[CV] END .....C=10, degree=4, kernel=poly; total time= 1.8s
Best Hyperparameters: {'C': 10, 'degree': 2, 'kernel': 'poly'}
Best Model Accuracy on Test Set: 90.05%

```

Figure 2.14: Output

Αυτή η συμπεριφορά, είναι ερμηνεύσιμη, καθώς οι δύο νέες κλάσεις είναι πιο εύκολα διαχωρίσιμες, συγκριτικά με τις πρώτες, όπως καταλαβαίνουμε. Οι εικόνες σκύλων και γάτων, παρουσιάζουν πιθανώς περισσότερα κοινά στοιχεία, απότι αυτές των αυτοκινήτων με τους σκύλους. Για αυτόν λοιπόν τον λόγο, τα SVM μπορούν να ταξινομήσουν τις εικόνες με μεγαλύτερη ακρίβεια! Παρακάτω παρατίθενται οι εικόνες των learning curves και του confusion matrix του SVM με παραμέτρους $C = 10$, $\text{degree} = 2$, $\text{kernel} = \text{poly}$, το οποίο προέκυψε από το πολυωνυμικό grid search.

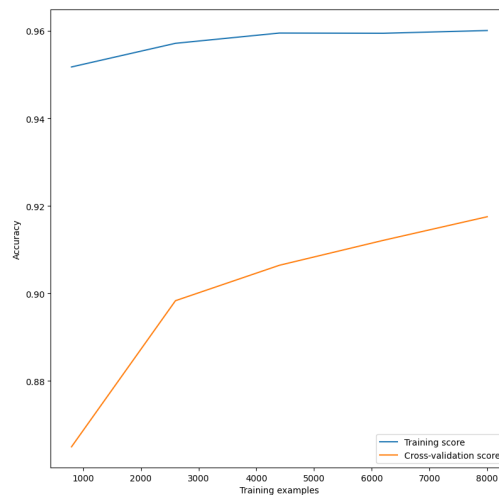


Figure 2.15: Learning Curves

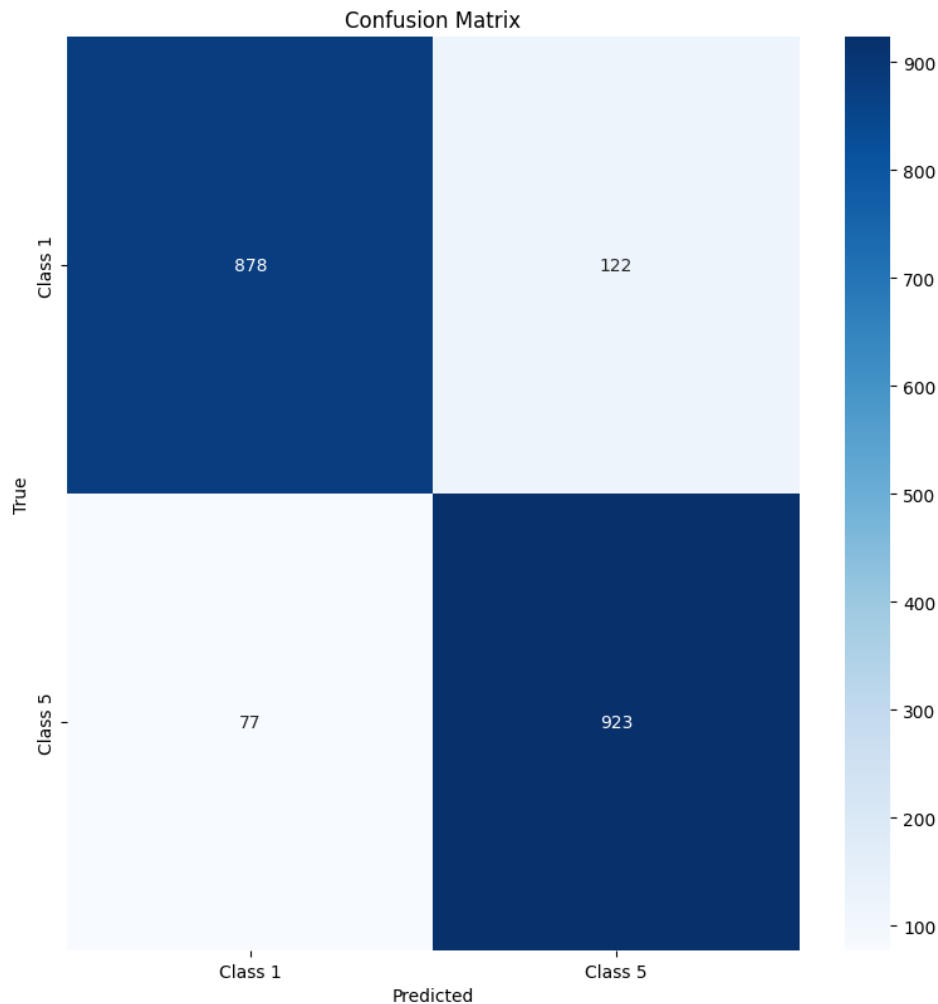


Figure 2.16: Confusion Matrix

2.6 Σύγκριση του SVM με τους κατηγοριοποιητές KNN και KNC

Στην συγκεκριμένη ενότητα, χρειάστηκε να γίνουν κάποιες τροποποιήσεις στον κώδικα των κατηγοριοποιητών KNN και KNC, ώστε να ταξινομούν και αυτοί μόνο τις κλάσεις 3 και 5 της Cifar10 και να είναι αξιόπιστη η σύγκριση μας.

2.6.1 K-Nearest Neighbor Classifier

Ο κατηγοριοποιητής K πλησιέστερου γείτονα, είναι ένας διάσημος αλγόριθμος μηχανικής μάθησης, ο οποίος μπορεί να διαχειριστεί προβλήματα regression και classification. Κατά την διάρκεια της φάσης εκπαίδευσης, ο KNN αποθηκεύει το dataset σαν αναφορά. Όταν έρχεται η ώρα να κάνει προβλέψεις, με βάση μια συγκεκριμένη ιδιότητα που θεωρεί ως απόσταση (συνήθως την ευκλείδεια απόσταση), υπολογίζει την απόσταση ανάμεσα στα δεδομένα που του δίνουμε ως input, και όλα τα παραδείγματα εκπαίδευσης. Έπειτα, ο αλγόριθμος ταυτοποιεί τους K (στην περίπτωση μας 1, 3) πλησιέστερους γείτονες στο input.

Αν το πρόβλημα μας είναι κατηγοριοποίησης ο αλγόριθμος αναθέτει την πιο συχνή κλάση που συναντάει στους γείτονες ως την πρόβλεψη του για το input data.

2.6.2 Nearest Centroid Classifier

Ο κατηγοριοποιητής πλησιέστερου κέντρου, είναι και αυτός ένας αλγόριθμος μηχανικής μάθησης, τον οποίο μπορούμε να εφαρμόσουμε σε προβλήματα κατηγοριοποίησης. Η κάθε κλάση του dataset, αναπαριστάται από το κέντρο της (δηλαδή τον "μέσο όρο" μιας τιμής που επιλέξαμε). Έτσι, όταν περάσουμε στον κατηγοριοποιητή κάποιο input, αυτός θα το αναθέσει σύμφωνα με την κλάση στην οποία ανήκει το πλησιέστερο κέντρο από το input data.

Αφού υλοποίησα λοιπόν τους παραπάνω κατηγοριοποιητές στο περιβάλλον της python, συνέκρινα την ακρίβεια τους, και όπως είναι φανερό από τον πίνακα παρακάτω, ο κατηγοριοποιητής πλησιέστερου κέντρου, είναι ο πιο αποτελεσματικός και ακριβής, ενώ ακολουθούν με την σειρά ο κατηγοριοποιητής πλησιέστερου γείτονα για $k = 1$, και ο κατηγοριοποιητής πλησιέστερου γείτονα για $k = 3$

Αλγόριθμος	Accuracy
KNN ($k = 1$)	57.50%
KNN ($k=3$)	56.90%
KNC	57.95%

Όπως φαίνεται, οι κατηγοριοποιητές της ενδιάμεσης εργασίας, έχουν αυξημένη ακρίβεια όταν πρόκειται να ταξινομήσουν μόνο δύο κλάσεις. Πλησιάζουν σε μια απόσταση της τάξης του 10% την ακρίβεια των SVM, τα οποία βέβαια όπως ήταν αναμενόμενο είναι πιο αποτελεσματικά στην κατηγοριοποίηση των δεδομένων! Αυτό μας δείχνει ότι τα συγκεκριμένα δεδομένα "ταιριάζουν" πολύ στον τύπο αυτών των κατηγοριοποιητών. Η αύξηση στην ακρίβεια ήταν βέβαια αναμενόμενη με την μείωση των πιθανών κλάσεων, μιας και η "απλότητα" των δύο αυτών κατηγοριοποιητών, τους καθιστά πιο αποτελεσματικούς σε λιγότερο σύνθετα προβλήματα.

Εδώ έχει τελειώσει η εργασία!!!

Παρολαυτά, παραθέτω κάποια κομμάτια από μια προηγούμενη μορφή που είχε η εργασία μου, διότι ,δυστυχώς, τελευταία στιγμή (αναλογικά με τον χρόνο που είχαμε) ανακάλυψα ένα λάθος στον κώδικα μου, το οποίο είχε ως αποτέλεσμα να μην "φορτώνονται" ολόκληρα τα δεδομένα στο SVM. Αυτό οδήγησε σε μειωμένες αποδόσεις των μοντέλων μου, και σε παράξενες συμπεριφορές όσον αφορά το χρόνο που χρειαζότανε ο κώδικας για να τρέξει. Λόγω ελλειψής εμπειρίας όμως δεν το συνειδητοποίησα άμεσα. Επειδή όμως μερικά κομμάτια όπως θα δείτε παρακάτω πήραν αρκετό χρόνο για να διερευνηθούν, τα επισυνάπτω σε μια πολύ πρόχειρη μορφή, για να τα έχετε υπόψιν σας.

Για αυτό τον λόγο δεν μελετήθηκε πολύ ο γραμμικός πυρήνας στην τελική εργασία, καθώς επαίρνε αρκετή ώρα, η οποία ήταν πολύτιμη και είχε δυστυχώς αφιερωθεί στην προηγούμενη, λανθασμένη υλοποίηση

Ευχαριστώ πολύ για την κατανόηση!

Στην συνέχεια, θα πραγματοποιήσουμε ένα grid search, όπου θα εξετάσουμε τους πιθανούς συνδυασμούς διαφόρων τιμών του C και τριών συναρτήσεων πυρήνων. Πιο συγκεκριμένα, το πλέγμα θα δημιουργηθεί με αυτές τις παραμέτρους:

```
# Grid Search
parameters_grid = {'kernel': ['linear', 'poly', 'rbf'],
                   'C': [0.05, 0.1, 1, 10]}

grid_search = GridSearchCV(SVC(random_state=42), parameters_grid, cv=3, scoring='accuracy', verbose = 2)

# Measure grid search time
start_time = time.time()
grid_search.fit(X_train_binary, y_train_binary)
end_time = time.time()

# Get the best hyperparameters
best_params = grid_search.best_params_
print("Best Hyperparameters:", best_params)
```

Figure 2.17: Grid Search

Όπως βλέπουμε από τα αποτελέσματα, ο κώδικας μας "δοκιμάζει" τους διάφορους συνδυασμούς (χρησιμοποιώντας μάλιστα και την τεχνική cross validation), και μας προτείνει σαν βέλτιστες τιμές των υπερπαραμέτρων, το C να έχει την τιμή 10 και συνάρτηση πυρήνα την RBF. Με αυτό τον συνδυασμό, το SVM μας έχει ποσοστό ακρίβειας στο test set ίσο με 56.30%. Στην εικόνα που παρατίθεται, μπορούμε επίσης να δούμε πόσο χρόνο χρειάστηκε ο κάθε επιμέρους συνδυασμός, καθώς και συνολικά το grid search

```

Fitting 3 folds for each of 12 candidates, totalling 36 fits
[CV] END .....C=0.05, kernel=linear; total time= 2.1s
[CV] END .....C=0.05, kernel=linear; total time= 2.1s
[CV] END .....C=0.05, kernel=linear; total time= 2.1s
[CV] END .....C=0.05, kernel=poly; total time= 2.1s
[CV] END .....C=0.05, kernel=poly; total time= 2.0s
[CV] END .....C=0.05, kernel=poly; total time= 2.0s
[CV] END .....C=0.05, kernel=rbf; total time= 3.3s
[CV] END .....C=0.05, kernel=rbf; total time= 3.4s
[CV] END .....C=0.05, kernel=rbf; total time= 3.3s
[CV] END .....C=0.1, kernel=linear; total time= 2.4s
[CV] END .....C=0.1, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, kernel=linear; total time= 2.4s
[CV] END .....C=0.1, kernel=poly; total time= 2.1s
[CV] END .....C=0.1, kernel=poly; total time= 2.1s
[CV] END .....C=0.1, kernel=poly; total time= 2.1s
[CV] END .....C=0.1, kernel=rbf; total time= 3.4s
[CV] END .....C=0.1, kernel=rbf; total time= 3.4s
[CV] END .....C=0.1, kernel=rbf; total time= 3.3s
[CV] END .....C=1, kernel=linear; total time= 4.6s
[CV] END .....C=1, kernel=linear; total time= 4.6s
[CV] END .....C=1, kernel=linear; total time= 4.5s
[CV] END .....C=1, kernel=poly; total time= 2.4s
[CV] END .....C=1, kernel=poly; total time= 2.4s
[CV] END .....C=1, kernel=poly; total time= 2.4s
[CV] END .....C=1, kernel=rbf; total time= 3.3s
[CV] END .....C=1, kernel=rbf; total time= 3.3s
[CV] END .....C=1, kernel=rbf; total time= 3.3s
[CV] END .....C=10, kernel=linear; total time= 25.0s
[CV] END .....C=10, kernel=linear; total time= 22.8s
[CV] END .....C=10, kernel=linear; total time= 22.2s
[CV] END .....C=10, kernel=poly; total time= 4.8s
[CV] END .....C=10, kernel=poly; total time= 4.3s
[CV] END .....C=10, kernel=poly; total time= 4.5s
[CV] END .....C=10, kernel=rbf; total time= 3.4s
[CV] END .....C=10, kernel=rbf; total time= 3.4s
[CV] END .....C=10, kernel=rbf; total time= 3.4s
Best Hyperparameters: {'C': 10, 'kernel': 'rbf'}
Best Model Accuracy on Test Set: 56.30%
Grid Search time: 175.34 seconds

```

Figure 2.18: Grid Results

Στην συνέχεια, θα πραγματοποιήσουμε ένα grid search, όπου θα εξετάσουμε τους πιθανούς συνδυασμούς διαφόρων τιμών του C και τριών συναρτήσεων πυρήνων. Πιο συγκεκριμένα, το πλέγμα θα δημιουργηθεί με αυτές τις παραμέτρους:

```

# Grid Search
parameters_grid = {'kernel': ['linear', 'poly', 'rbf'],
                  'C': [0.05, 0.1, 1, 10]}

grid_search = GridSearchCV(SVC(random_state=42), parameters_grid, cv=3, scoring='accuracy', verbose = 2)

# Measure grid search time
start_time = time.time()
grid_search.fit(X_train_binary, y_train_binary)
end_time = time.time()

# Get the best hyperparameters
best_params = grid_search.best_params_
print("Best Hyperparameters:", best_params)

```

Figure 2.19: Grid Search

Όπως βλέπουμε από τα αποτελέσματα, ο κώδικας μας "δοκιμάζει" τους διάφορους συνδυασμούς (χρησιμοποιώντας μάλιστα και την τεχνική cross validation), και μας προτείνει σαν βέλτιστες τιμές των υπερπαραμέτρων, το C να έχει την τιμή 10 και συνάρτηση πυρήνα την RBF. Με αυτό τον συνδυασμό, το SVM μας έχει ποσοστό ακρίβειας στο test set ίσο με 56.30%.

Στην εικόνα που παρατίθεται, μπορούμε επίσης να δούμε πόσο χρόνο χρειάστηκε ο κάθε επιμέρους συνδυασμός, καθώς και συνολικά το grid search

```
Fitting 3 folds for each of 12 candidates, totalling 36 fits
[CV] END .....C=0.05, kernel=linear; total time= 2.1s
[CV] END .....C=0.05, kernel=linear; total time= 2.1s
[CV] END .....C=0.05, kernel=linear; total time= 2.1s
[CV] END .....C=0.05, kernel=poly; total time= 2.1s
[CV] END .....C=0.05, kernel=poly; total time= 2.0s
[CV] END .....C=0.05, kernel=poly; total time= 2.0s
[CV] END .....C=0.05, kernel=rbf; total time= 3.3s
[CV] END .....C=0.05, kernel=rbf; total time= 3.4s
[CV] END .....C=0.05, kernel=rbf; total time= 3.3s
[CV] END .....C=0.1, kernel=linear; total time= 2.4s
[CV] END .....C=0.1, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, kernel=linear; total time= 2.4s
[CV] END .....C=0.1, kernel=poly; total time= 2.1s
[CV] END .....C=0.1, kernel=poly; total time= 2.1s
[CV] END .....C=0.1, kernel=poly; total time= 2.1s
[CV] END .....C=0.1, kernel=rbf; total time= 3.4s
[CV] END .....C=0.1, kernel=rbf; total time= 3.4s
[CV] END .....C=0.1, kernel=rbf; total time= 3.3s
[CV] END .....C=1, kernel=linear; total time= 4.6s
[CV] END .....C=1, kernel=linear; total time= 4.6s
[CV] END .....C=1, kernel=linear; total time= 4.5s
[CV] END .....C=1, kernel=poly; total time= 2.4s
[CV] END .....C=1, kernel=poly; total time= 2.4s
[CV] END .....C=1, kernel=poly; total time= 2.4s
[CV] END .....C=1, kernel=rbf; total time= 3.3s
[CV] END .....C=1, kernel=rbf; total time= 3.3s
[CV] END .....C=1, kernel=rbf; total time= 3.3s
[CV] END .....C=10, kernel=linear; total time= 25.0s
[CV] END .....C=10, kernel=linear; total time= 22.8s
[CV] END .....C=10, kernel=linear; total time= 22.2s
[CV] END .....C=10, kernel=poly; total time= 4.8s
[CV] END .....C=10, kernel=poly; total time= 4.3s
[CV] END .....C=10, kernel=poly; total time= 4.5s
[CV] END .....C=10, kernel=rbf; total time= 3.4s
[CV] END .....C=10, kernel=rbf; total time= 3.4s
[CV] END .....C=10, kernel=rbf; total time= 3.4s
Best Hyperparameters: {'C': 10, 'kernel': 'rbf'}
Best Model Accuracy on Test Set: 56.30%
Grid Search time: 175.34 seconds
```

Figure 2.20: Grid Results

Στην συνέχεια της μελέτης των SVM μας, αποφάσισα να υλοποιήσω ένα λίγο πιο σύνθετο grid search. Πιο συγκεκριμένα, εκτός από την παράμετρο C και τις συναρτήσεις πυρήνα, το πλέγμα μας αποτελούνταν και από δύο νέες παραμέτρους, την παράμετρο degree που αφορά το polynomial kernel, καθώς και την gamma που συνδέεται με το RBF kernel. Όταν όμως επιχειρήσα να τρέξω τον κώδικα, συνειδητοποίησα ότι χρειαζόταν ένας πολύ μεγάλος αριθμός ωρών και ημερών προκειμένου να ολοκληρωθεί. Για αυτόν λοιπόν τον λόγο, θα εξετάσουμε τις παραμέτρους αυτές ξεχωριστά, με σταθεροποιημένες την παράμετρο C αλλά και την συνάρτηση πυρήνα. Ενδεικτικά, παρατίθεται μια εικόνα του κώδικα που υλοποιεί το πλέγμα, και μια ακόμα λίγο πριν διακόψω το πρόγραμμα του σύνθετου grid search.

```
# Grid Search
parameters_grid = {'kernel': ['linear', 'poly', 'rbf'],
                   'C': [0.1, 1, 10],
                   'degree': [2, 3, 4], # Polynomial degree
                   'gamma': ['scale', 'auto', 0.1, 1.0]} # RBF kernel parameter
grid_search = GridSearchCV(SVC(random_state=42), parameters_grid, cv=3, scoring='accuracy', verbose = 2)
```

Figure 2.21: Code

```
Fitting 3 folds for each of 54 candidates, totalling 162 fits
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=poly; total time= 2.5s
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=poly; total time= 2.6s
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=poly; total time= 2.7s
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=rbf; total time= 3.2s
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=rbf; total time= 3.2s
[CV] END .....C=0.1, degree=2, gamma=0.1, kernel=rbf; total time= 3.2s
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=linear; total time= 2.2s
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=poly; total time= 1.6min
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=poly; total time= 1.3min
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=poly; total time= 1.3min
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=rbf; total time= 3.4s
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=rbf; total time= 3.4s
[CV] END .....C=0.1, degree=2, gamma=1.0, kernel=rbf; total time= 3.4s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=poly; total time= 32.5s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=poly; total time= 30.4s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=poly; total time= 30.1s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=rbf; total time= 3.3s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=rbf; total time= 3.3s
[CV] END .....C=0.1, degree=3, gamma=0.1, kernel=rbf; total time= 3.3s
[CV] END .....C=0.1, degree=3, gamma=1.0, kernel=linear; total time= 2.4s
[CV] END .....C=0.1, degree=3, gamma=1.0, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=3, gamma=1.0, kernel=linear; total time= 2.3s
[CV] END .....C=0.1, degree=3, gamma=1.0, kernel=poly; total time=793.9min
```

Figure 2.22: Output

Θα μελετήσουμε λίγο ακόμα την συνάρτηση πυρήνα RBF, μιας και αυτή μας έδωσε τα καλύτερα αποτελέσματα όπως είδαμε παραπάνω. Το kernel RBF, έχει κάποια ακόμα παράμετρο την οποία μπορούμε να διερευνήσουμε, την παράμετρο γ (gamma). Η παράμετρος αυτή, καθορίζει το πόσο "ελαστικό" είναι το πεδίο επιδόσεων του πυρήνα. Επηρεάζει τον τρόπο με τον οποίο το SVM προσαρμόζει τις αποφάσεις αποκλεισμού στα δεδομένα εκπαίδευσης. Ειδικότερα

- Χαμηλή τιμή του γ : Εκτείνει το πεδίο επιδόσεων και καθιστά τον πυρήνα RBF πιο "ελαστικό", με αποτέλεσμα το μοντέλο να είναι λιγότερο ευαίσθητο στις λεπτομέρειες των δεδομένων εκπαίδευσης.
- Υψηλή τιμή του γ : Σφίγγει το πεδίο επιδόσεων και καθιστά τον πυρήνα RBF πιο "σφιχτό", με αποτέλεσμα το μοντέλο να είναι πιο ευαίσθητο στις λεπτομέρειες και τον θόρυβο των δεδομένων εκπαίδευσης.

Θα προσπαθήσουμε λοιπόν να εξετάσουμε την επίδραση της παραμέτρου αυτής στο SVM μας, και να βρούμε την βέλτιστη τιμή που μπορεί να πάρει. Για να το καταφέρουμε αυτό, θα υλοποιήσουμε μια μικρή κλίμακας grid search.

```
# Grid Search
parameters_grid = {'kernel': [ 'rbf'],
                   'C': [1], 'gamma':[0.05, 0.1, 0.5, 1.0, 2.0]}
grid_search = GridSearchCV(SVC(random_state=42), parameters_grid, cv=3, scoring='accuracy', verbose = 2)

# Measure grid search time
start_time = time.time()
grid_search.fit(X_train_binary, y_train_binary)
end_time = time.time()

# Get the best hyperparameters
best_params = grid_search.best_params_
print("Best Hyperparameters:", best_params)

# Get the best model
best_svm_classifier = grid_search.best_estimator_
```

Figure 2.23: Grid Search

Κρατάμε σταθερή την παράμετρο C και ίση με 1, και αλλάζουμε τις τιμές του gamma. Δοκιμάζουμε τις τιμές 0.05, 0.1, 0.5, 1.0, 2.0 αντίστοιχα.

```
Fitting 3 folds for each of 5 candidates, totalling 15 fits
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.3s
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.3s
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.3s
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=0.5, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=1.0, kernel=rbf; total time= 3.8s
[CV] END .....C=1, gamma=1.0, kernel=rbf; total time= 3.8s
[CV] END .....C=1, gamma=1.0, kernel=rbf; total time= 3.8s
[CV] END .....C=1, gamma=2.0, kernel=rbf; total time= 4.3s
[CV] END .....C=1, gamma=2.0, kernel=rbf; total time= 4.2s
[CV] END .....C=1, gamma=2.0, kernel=rbf; total time= 4.2s
Best Hyperparameters: {'C': 1, 'gamma': 0.05, 'kernel': 'rbf'}
Best Model Accuracy on Test Set: 57.25%
```

Figure 2.24: Results

Όπως φαίνεται από τα αποτελέσματα, το SVM μας αποκτά την μέγιστη ακρίβεια 57.25% για $\gamma = 0.05$. Εφόσον είναι η ελάχιστη τιμή που είχαμε στο grid μας, αποφάσισα να φτιάξω άλλο ένα πλέγμα στο οποίο θα ξεκινάμε από μια μικρότερη τιμή γ και θα φτάνουμε μέχρι την τιμή 0.05 προκειμένου να βρεθεί ενδεχομένως μια καλύτερη τιμή. Πιο συγκεκριμένα δοκιμάστηκαν οι τιμές $\gamma = [0.009, 0.01, 0.02, 0.03, 0.04, 0.05]$. Το αποτέλεσμα του grid search φαίνεται να δείχνει πως το βέλτιστο γ είναι η τιμή 0.04. Παρόλαυτα, η ακρίβεια δεν ξεπερνάει αυτήν του προηγούμενου grid search, επομένως συμπεραίνουμε με μια επιφύλαξη ότι η ιδανικότερη τιμή του γ είναι 0.05!

```
# Grid Search
parameters_grid = {'kernel': ['poly'],
                   'C': [1, 10],
                   'degree': [2, 3, 4]}
grid_search = GridSearchCV(SVC(random_state=42), parameters_grid, cv=3, scoring='accuracy', verbose = 2)
```

Figure 2.26: Poly Grid

```
Fitting 3 folds for each of 6 candidates, totalling 18 fits
[CV] END .....C=1, gamma=0.009, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.009, kernel=rbf; total time= 3.3s
[CV] END .....C=1, gamma=0.009, kernel=rbf; total time= 3.3s
[CV] END .....C=1, gamma=0.01, kernel=rbf; total time= 3.3s
[CV] END .....C=1, gamma=0.01, kernel=rbf; total time= 3.3s
[CV] END .....C=1, gamma=0.01, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.02, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.02, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.02, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.03, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.03, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.03, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.04, kernel=rbf; total time= 3.5s
[CV] END .....C=1, gamma=0.04, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.04, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.4s
[CV] END .....C=1, gamma=0.05, kernel=rbf; total time= 3.4s
Best Hyperparameters: {'C': 1, 'gamma': 0.04, 'kernel': 'rbf'}
Best Model Accuracy on Test Set: 56.85%
```

Figure 2.25: Results 2

Θα πραγματοποιήσουμε την ίδια διαδικασία για το polynomial kernel και την παράμετρο degree αντίστοιχα. Αυτή, καθορίζει τον βαθμό του πολυωνυμικού πυρήνα. Ο πυρήνας πολυωνύμου έχει τη μορφή: $K(x, y) = (x * y + c)^d$ όπου x, y είναι τα δύο διανύσματα εισόδου, και c είναι μια σταθερά. Το d είναι ο βαθμός του πολυωνύμου. Όσο μεγαλύτερη η τιμή του degree, τόσο πιο υψηλή είναι η τάξη του πολυωνύμου και τόσο πιο πολύπλοκος είναι ο χώρος των χαρακτηριστικών που το μοντέλο μπορεί να αντιπροσωπεύσει. Ωστόσο, υψηλότερες τιμές του degree μπορεί να οδηγήσουν σε μοντέλα που χρειάζονται περισσότερα δεδομένα εκπαίδευσης και είναι πιο επιρρεπή στην υπερεκπαίδευση. Το πλέγμα που θα υλοποιήσουμε φαίνεται παρακάτω. Δοκιμάζουμε δύο τιμές της C και τρεις διαφορετικές τιμές του degree.

Από τα αποτελέσματα, είναι ξεκάθαρο ότι το συγκεκριμένο kernel δεν έχει την ίδια αποτελεσματικότητα στο συγκεκριμένο dataset, και αποδεικνύεται, ότι το RBF είναι όντως η ιδανική συνάρτηση πυρήνα για το SVM μας.

```

Fitting 3 folds for each of 6 candidates, totalling 18 fits
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=2, kernel=poly; total time= 2.1s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.3s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.3s
[CV] END .....C=1, degree=3, kernel=poly; total time= 2.4s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.6s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.5s
[CV] END .....C=1, degree=4, kernel=poly; total time= 2.5s
[CV] END .....C=10, degree=2, kernel=poly; total time= 2.5s
[CV] END .....C=10, degree=2, kernel=poly; total time= 2.7s
[CV] END .....C=10, degree=2, kernel=poly; total time= 2.5s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.7s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.3s
[CV] END .....C=10, degree=3, kernel=poly; total time= 4.5s
[CV] END .....C=10, degree=4, kernel=poly; total time= 5.9s
[CV] END .....C=10, degree=4, kernel=poly; total time= 5.9s
[CV] END .....C=10, degree=4, kernel=poly; total time= 5.6s
Best Hyperparameters: {'C': 10, 'degree': 2, 'kernel': 'poly'}
Best Model Accuracy on Test Set: 54.00%
Grid Search time: 65.06 seconds

```

Figure 2.27: Poly Results